

Лабораторная работа №5

Редактирование и удаление данных профиля

О чем лабораторная?

При повороте устройства Android пересоздает экран профиля, поэтому введенный, но еще не сохраненный текст может исчезнуть. В этой работе используем `ViewModel`, чтобы сохранить состояние экрана при повороте, а также добавим удаление профиля.

Главное, что нужно понять

- поворот экрана пересоздает `Activity`
 - `ViewModel` хранит данные, пока существует экран, и переживает пересоздание `Activity`
 - `ViewModel` не заменяет `SharedPreferences`: первый хранит временное состояние экрана, второй - данные между запусками приложения
 - изменение полей ввода нужно передавать в `ViewModel` сразу, а не только по кнопке "Сохранить"
-

Часть 1. Цель работы

Научиться сохранять данные при повороте экрана с помощью `ViewModel`, реализовать редактирование и удаление профиля пользователя

Часть 2. Почему нужен ViewModel

В прошлой лабораторной мы научились сохранять данные в `SharedPreferences` и создали два класса-помощника: `ProfilePreferences` для данных профиля и `AppPreferences` для флага `OnBoard`. Но есть одна проблема, с которой вы могли столкнуться: если повернуть телефон, когда на экране открыт профиль, все введенные данные в полях исчезают.

Это происходит потому, что при повороте экрана Android пересоздает `Activity`. Все данные, которые хранились в полях активности, теряются.

Для решения этой проблемы в Android есть `ViewModel` - объект, который переживает поворот экрана и хранит данные, пока жив экран.

Часть 3. Что нужно сделать

1. Добавить зависимость `androidx.lifecycle:lifecycle-viewmodel`
 2. Создать `ProfileViewModel` для хранения данных профиля
 3. Интегрировать `ViewModel` с `ProfileActivity`
 4. Реализовать редактирование и удаление профиля
 5. Проверить, что данные сохраняются при повороте экрана
-

Часть 4. Практическая часть

Шаг 1. Добавьте зависимость

Откройте `build.gradle` (Module: `app`) и добавьте в раздел `dependencies`:

```
implementation('androidx.lifecycle:lifecycle-viewmodel:2.6.2')
```

Синхронизируйте проект.

Шаг 2. Создайте `ProfileViewModel`

Создайте новый класс `ProfileViewModel`:

```
package com.example.avito;

import androidx.lifecycle.ViewModel;

public class ProfileViewModel extends ViewModel {

    private String username;
    private String email;
    private String aboutMe;

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getAboutMe() {
        return aboutMe;
    }

    public void setAboutMe(String aboutMe) {
        this.aboutMe = aboutMe;
    }
}
```

Разберем, что здесь происходит:

- `ViewModel` - это специальный класс, который живет дольше, чем `Activity`. При повороте экрана `Activity` пересоздается, но `ViewModel` остается

- Мы храним данные профиля в полях `ViewModel`, а не в `Activity`. Когда `Activity` пересоздается, она получает те же данные из `ViewModel`

Шаг 3. Интегрируйте `ViewModel` с `ProfileActivity`

Измените `ProfileActivity`:

```
public class ProfileActivity extends AppCompatActivity {

    private EditText etUsername;
    private EditText etEmail;
    private EditText etAboutMe;
    private ProfilePreferences profilePrefs;
    private ProfileViewModel viewModel;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_profile);

        etUsername = findViewById(R.id.et_username);
        etEmail = findViewById(R.id.et_email);
        etAboutMe = findViewById(R.id.et_about_me);
        Button btnSave = findViewById(R.id.btn_save);
        Button btnDelete = findViewById(R.id.btn_delete);

        profilePrefs = new ProfilePreferences(this);

        // Получаем ViewModel
        viewModel = new ViewModelProvider(this).get(ProfileViewModel.class);

        // Загружаем данные из ViewModel или из SharedPreferences
        if (viewModel.getUsername() != null) {
            etUsername.setText(viewModel.getUsername());
            etEmail.setText(viewModel.getEmail());
            etAboutMe.setText(viewModel.getAboutMe());
        } else {
            etUsername.setText(profilePrefs.getUsername());
            etEmail.setText(profilePrefs.getEmail());
            etAboutMe.setText(profilePrefs.getAboutMe());
        }

        // сохраняем КАЖДОЕ изменение в ViewModel
        // поэтому введенный текст не исчезнет при повороте до нажатия "Сохранить"
        TextWatcher profileTextWatcher = new SimpleTextWatcher() {
            @Override
            public void afterTextChanged(Editable editable) {
                viewModel.setUsername(etUsername.getText().toString());
                viewModel.setEmail(etEmail.getText().toString());
                viewModel.setAboutMe(etAboutMe.getText().toString());
            }
        };
    }
};
```

```

etUsername.addTextChangedListener(profileTextWatcher);
etEmail.addTextChangedListener(profileTextWatcher);
etAboutMe.addTextChangedListener(profileTextWatcher);

// кнопка сохранения
btnSave.setOnClickListener(v -> {
    viewModel.setUsername(etUsername.getText().toString());
    viewModel.setEmail(etEmail.getText().toString());
    viewModel.setAboutMe(etAboutMe.getText().toString());

    profilePrefs.saveProfile(
        viewModel.getUsername(),
        viewModel.getEmail(),
        viewModel.getAboutMe()
    );
    Toast.makeText(this, "Профиль сохранен", Toast.LENGTH_SHORT).show();
});

// кнопка удаления профиля
btnDelete.setOnClickListener(v -> {
    viewModel.setUsername("");
    viewModel.setEmail("");
    viewModel.setAboutMe("");

    profilePrefs.clearProfile();
    etUsername.setText("");
    etEmail.setText("");
    etAboutMe.setText("");
    Toast.makeText(this, "Профиль удален", Toast.LENGTH_SHORT).show();
});
}
}

```

Чтобы не писать пустые методы `TextWatcher` три раза, создайте рядом с `ProfileActivity` вспомогательный класс:

```

public abstract class SimpleTextWatcher implements TextWatcher {
    @Override
    public void beforeTextChanged(CharSequence text, int start, int count, int
after)
    { }

    @Override
    public void onTextChanged(CharSequence text, int start, int before, int count)
    { }
}

```

Шаг 4. Добавьте кнопку удаления

В `activity_profile.xml` добавьте кнопку "Удалить профиль":

```
<Button
    android:id="@+id/btn_delete"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Удалить профиль" />
```

Шаг 5. Проверка результата

1. Запустите приложение
2. Откройте экран профиля, введите данные
3. Поверните телефон (нажмите кнопку поворота в эмуляторе)
4. Убедитесь, что данные в полях не пропали
5. Нажмите "Сохранить", полностью закройте приложение и откройте снова
6. Убедитесь, что данные загрузились из `SharedPreferences`
7. Нажмите "Удалить профиль" и убедитесь, что поля очистились

Базовый уровень

Выполните все шаги практической части. До нажатия кнопки «Сохранить» введите данные, поверните устройство и убедитесь, что текст в полях не исчез.

Дополнительно

1. Добавьте в `ProfileViewModel` поле `notificationsEnabled` (тип `boolean`) и методы `getNotificationsEnabled()` / `setNotificationsEnabled(boolean)`. Сохраняйте его в `SharedPreferences` и загружайте при открытии экрана
2. Добавьте подтверждение удаления: перед удалением профиля показывайте `AlertDialog` с вопросом "Вы уверены, что хотите удалить профиль?" и кнопками "Да" и "Нет"

Требования к итоговой версии

- `ProfileViewModel` хранит данные профиля и переживает поворот экрана
- `ProfileActivity` использует `ViewModel` для хранения данных
- Данные сохраняются в `SharedPreferences` при нажатии "Сохранить"
- Данные загружаются из `SharedPreferences` при первом открытии экрана
- Кнопка "Удалить профиль" очищает поля и удаляет данные из `SharedPreferences`
- Данные в полях не пропадают при повороте экрана

Дополнительное задание

Добавьте валидацию полей перед сохранением (как в дополнительном задании ЛР-04), но выполняйте ее в `ProfileViewModel`. Создайте метод `validate()`, который возвращает `true`, если все поля заполнены корректно, и `false` в противном случае. В `ProfileActivity` проверяйте результат перед сохранением